



**Design and Implementation of an eBPF-based Runtime Security
Monitoring Tool**

Candidato: Simone Romano (s344024)
Relatore: Prof. Riccardo Sisto
Tutor aziendale: Dr. David Soldani
Anno accademico: 2025–2026

Riassunto

Quadro generale e obiettivo. La crescente adozione di container e piattaforme di orchestrazione rende necessario osservare il comportamento delle applicazioni senza modificarne il codice e senza dipendere esclusivamente dai log prodotti in user space. eBPF permette di collegare programmi verificati a punti specifici del kernel Linux, offrendo visibilità su processi, credenziali, file, memoria e meccanismi di isolamento. L'obiettivo della tesi è progettare e implementare un prototipo di monitoraggio runtime basato su eBPF, capace di raccogliere eventi host-level in modo selettivo e di conservarne il significato fino alla successiva analisi in user space.

Il lavoro considera Rocky Linux 8.10 con kernel 4.18 come ambiente di riferimento. La compatibilità viene verificata sul kernel effettivo, includendo le estensioni distribuite dal vendor, poiché CO-RE e BTF adattano l'accesso alle strutture ma non rendono disponibili hook, helper o meccanismi di attachment assenti. Il sistema è uno strumento di osservazione e rilevamento: non modifica le decisioni del kernel e non agisce come motore di enforcement.

Implementazione degli hook nel sistema host. La parte principale del lavoro riguarda la selezione e l'implementazione degli hook necessari a rappresentare le operazioni osservate. I programmi eBPF sono collegati a syscall, tracepoint, funzioni del kernel e interfacce Linux Security Module. Questi punti non sono equivalenti: una syscall in ingresso conserva la richiesta del processo, un hook LSM espone un'operazione durante la mediazione di sicurezza, un tracepoint può confermare una transizione completata e un hook di ritorno aggiunge il risultato disponibile al chiamante.

Due forme di costruzione vengono riutilizzate dalle diverse famiglie. Gli hook diretti producono e inviano l'evento in una sola invocazione. Le operazioni che richiedono sia argomenti sia risultato usano invece programmi accoppiati: il programma di ingresso salva dati temporanei per il thread, mentre quello di ritorno recupera tali dati, aggiunge il valore restituito e completa l'evento. Workspace e scratch buffer per-CPU evitano di collocare record di grandi dimensioni nello stack eBPF. Gli argomenti sono serializzati con indici e limiti espliciti, quindi soltanto la parte popolata del record attraversa il perf-event buffer.

Gli hook di processo distinguono la richiesta di creazione dal ciclo di vita effettivo del task. Gli eventi relativi a fork, vfork e clone conservano il risultato della syscall e, per clone, anche il modello di condivisione richiesto. I tracepoint dello scheduler descrivono invece creazione, sostituzione dell'immagine eseguibile e terminazione. Per exec, gli ingressi di execve ed execveat mantengono percorso e argomenti del chiamante; gli hook *security_bprm* aggiungono l'identità risolta del binario; *sched_process_exec* conferma la transizione riuscita; un evento dedicato rappresenta il ritorno negativo.

Gli hook di identità separano analogamente richiesta, mediazione e stato installato. Le famiglie *setuid* e *setgid* registrano gli identificatori richiesti e il risultato. *security_task_fix_setuid* osserva la proposta durante il controllo di sicurezza, mentre *commit_creds* confronta le credenziali precedenti e nuove quando la sostituzione diventa attiva. Il relativo evento include UID, GID, namespace utente, secure bits e capability set. *cap_capable* rende infine visibili i singoli controlli di capability senza interpretarli come decisione finale sull'operazione.



Gli hook filesystem preservano la differenza tra il pathname fornito dal processo e l'oggetto risolto dal kernel. Le syscall `open`, `chmod` e `chown` descrivono intenzione e risultato, riunendo più varianti fisiche in uno schema logico comune. `security_file_open` riceve invece la struttura file già risolta e registra percorso kernel, device, inode e change time, insieme al pathname del chiamante quando recuperabile. Ulteriori hook osservano controlli di permesso, `ioctl`, `unlink`, `rename`, `symlink`, creazione di nodi speciali e modifiche alla topologia di mount.

La copertura host-level comprende inoltre `mmap`, `mprotect` e trasferimenti di memoria tra processi, nonché le transizioni di namespace richieste da `setns` e `unshare`. `switch_task_ns` registra soltanto gli identificatori effettivamente cambiati, mentre gli hook cgroup descrivono creazione, rimozione e spostamento dei task usando identità locali del kernel. Le superfici kernel-facing includono caricamento dei moduli, operazioni eBPF sottoposte a mediazione, registrazione di kprobe, risoluzione di simboli, creazione di entry procs, user-mode helper e letture di file avviate dal kernel. Tali eventi forniscono evidenza per i detector, ma non dimostrano da soli integrità o autorizzazione.

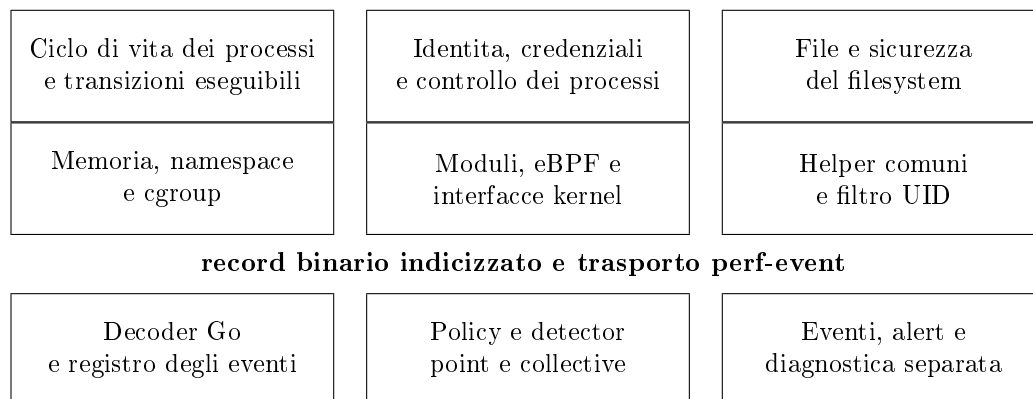


Figura 1: Famiglie di hook host-level e percorso comune verso l'analisi in user space.

Gli hook condividono inizializzazione, filtro UID, serializzazione indicizzata e sottomissione perf. Il contesto binario comune mantiene identità host e namespace-relative del processo, timestamp, syscall ed evento logico. Branch CO-RE espliciti gestiscono differenze in task, capability, inode e cgroup del kernel target. Loop e accessi variabili sono limitati in forma visibile al verifier; l'implementazione non approssima invece funzionalità non disponibili sul sistema osservato.

Elaborazione in user space. Un runtime sviluppato in Go seleziona i programmi necessari, espande le dipendenze interne e disabilita l'autoload dei programmi esclusi. Il decoder converte i record binari in eventi tipizzati secondo un registro condiviso. Le policy definiscono ambito e soppressione, mentre detector deterministici riconoscono condizioni point o brevi sequenze collective correlate per processo, risorsa file o cgroup. Gli alert conservano gli eventi che hanno completato il match e possono includere metadati MITRE ATT&CK come classificazione.

Validazione e risultati. Il prototipo è stato infine validato in un cluster Minikube single-node con monitor, target AES-beta e client in tre Pod distinti. Tre interazioni benigne non hanno prodotto alert; in tre esecuzioni dell'exploit è stata recuperata la flag sintetica e il detector ha ricostruito la sequenza composta da shell, esecuzione di `cat` e accesso riuscito al file protetto. Il test conferma la visibilità node-local del prototipo in questo scenario controllato, senza estendere la conclusione a monitoraggio cross-node, least privilege, copertura generale degli attacchi o enforcement.